

r.No	Theory Learning Outcomes (TLO's) aligned to CO's.	Learning content mapped with Theory Learning Outcomes (TLO's) and CO's.	Suggested Learning Pedagogies.
3	TLO 3.1 Justify the need of given scheduling criteria with relevant example. TLO 3.2 Explain with example the procedure of allocating CPU to the given process. TLO 3.3 Calculate turnaround time and average waiting time of the given scheduling algorithm. TLO 3.4 Explain functioning of the given necessary conditions leading to Deadlock.	Unit - III CPU Scheduling 3.1 Scheduling: Basic concept, CPU and I/O burst cycle 3.2 Preemptive and Non-preemptive scheduling, scheduling criteria 3.3 Types of Scheduling algorithms: First Come First Serve (FCFS), Shortest Job First (SJF), Shortest Remaining Time Next (SRTN), Round Robin (RR), Priority Scheduling, Multilevel Queue Scheduling 3.4 Deadlock: System Models, Necessary conditions Leading to Deadlock, Deadlock Handling: Deadlock prevention, Deadlock avoidance- Banker's Algorithm	Presentations Lecture Using Chalk-Board

CPU Scheduling

In multiprogramming system, the Operating Systems schedules the processes on the CPU to have maximum utilization of it and procedure is called CPU scheduling.

The Operating Systems uses various scheduling algorithm to schedule the processes.

Need Scheduling

- 1) Balanced Process mix: The scheduler selects a mix of CPU-bound (need more CPU) and I/O bound (need more I/O) processes to keep the CPU and I/O devices active.
- 2) Avoid Deadlocks: The OS ensures process don't get stuck waiting for resources by carefully managing resources allocation.
- 3) Dynamic Load Adjustments: The OS changes priorities based on the current state of system to avoid overloading one resource (CPU).
- 4) Fairness: Processes are scheduled fairly so no process is stuck waiting forever, preventing deadlocks & starvation.
- 5) Resource Optimization: The OS works to make the best use of CPU, memory, and I/O ensuring all resources are efficiently utilized.

DDDD

Question: Define CPU bound program
I/O bound program (2m)

CPU bound program

If execution of a program is highly dependent on CPU then it is known as CPU bound program.

or

If a task does a lot of operations using CPU it is called CPU-bound task.

I/O Bound program

If execution of program is dependent on input-output system and its resources such as disk drives and peripheral devices then it is known as I/O bound program.

Define CPU and I/O burst cycles (2m)

or

Describe I/O burst and CPU burst cycle with neat diagram (4m)

⇒ CPU burst cycle

It is time period when process is busy with CPU.

I/O burst cycle

It is time period when process is busy with I/O resources.

d) CPU burst cycle

Time period process is busy with the CPU (execution) is known as CPU burst cycle.

I/O burst cycle

Time period process is busy with the I/O devices (such as printer, scanner, peripheral devices) known as I/O burst cycle.

EXPLAIN ANY FOUR SCHEDULING CRITERIA (VIMP-4MARKS)

1) CPU utilization

In multiprogramming the main objective is to keep CPU as busy as possible. CPU utilization can range from 0 to 100 percent.

2) Throughput

It is number of processes that complete per unit time.

It measures of work done in the system.

Throughput depends on execution time required for any processes.

Simple words, its performance.

3) Turnaround time

Turnaround time is total time taken from the submission of process to its completion.

It includes all waiting, processing & I/O times.

$$\text{Turnaround time} = \text{Completion Time} - \text{Arrival Time}$$

4) Waiting Time

waiting time is total process spends in ready queue for CPU allocate.

It does not include the time spent executing or performing I/O operations.

$$\text{waiting time} = \text{Turnaround Time} - \text{Burst Time}$$

5) Response time

It is also known as completion time.

It is time between making a request & getting first reply. It doesn't mean that process is finished just that it has started responding. Some process may not finish.

Preemptive Scheduling

The CPU can be taken away from a running process.

- Allows higher priority processes to interrupt lower priority ones.

- Mostly preemptive scheduling used in Real time system where quick response is needed.

- Examples, Round Robin

Shortest Remaining Time First (SRTF)

Non-Preemptive Scheduling

- Once the process starts, it cannot be interrupted until it finishes or voluntarily gives up to CPU.

- Algorithms are - SJF (Shortest Job First)
FCFS (First Come First Serve)
Priority scheduling.

Question: How preemptive scheduling is better than non-preemptive scheduling.

⇒ 1) Pre-emptive scheduling allows shorter or more urgent process to interrupt long-running ones, leads to improving Response Time.

2) It reduces the time processes spend waiting in Queue by priority shortest & highest.

3) It also make CPU utilization efficiently, Better Real time systems,

4) Prevents Process monopolization.

- e) Following are the scheduling Criteria:
- ① Response Time
 - ② Waiting Time
 - ③ Turnaround Time
 - ④ ~~Throughput~~ Throughput
 - ⑤ CPU utilization

Homework : Compare preemtive and non preemtive process!

Question: Define Deadlock (2m)

A Deadlock is situation in operating system where two or more processes are unable to proceed because each one is waiting for resource that other holds.

A situation where a set of processes are blocked because each process is holding a resource and waiting for another resource that is held by another process.

This results in all involved processes stuck in infinite waiting state.

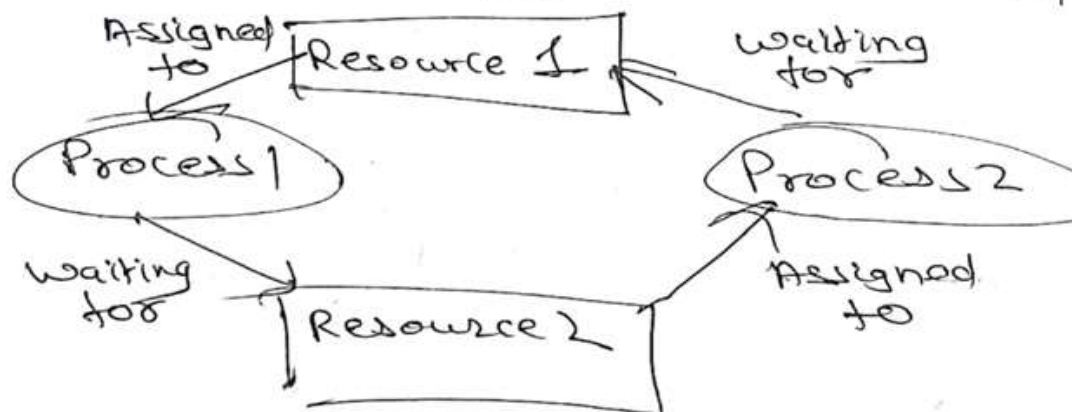
Example

Process P_1 holds Resource R_1

Process P_2 holds Resource R_2

Process P_1 waiting for Resource R_2

& Process P_2 waiting for Resource R_1



Necessary conditions for deadlock

d) Deadlock

Deadlock is a situation in which two or more processes are unable to proceed due to resources are held by other process.

This results in infinite waiting state of process.

Following are necessary conditions due to that deadlock occur.

1) Mutual Exclusion

Resources of each process or one of the process must be in non-shareable mode.

Only one process can share resource at a time.

2) Hold and wait

The process hold one or more resources and wait for additional resources held by the other processes.

3) No - Preemption

Process can't take resources forcibly from the other process.

They must be released voluntarily.

P.T.O

Page No. 7/12

1) Circular wait

There is circular chain of process where each process needs the resources hold by other process in chain.

Example, P_1, P_2, P_3 are process and R_1, R_2, R_3 are resources

P_1 Needs Resource hold by P_2

P_2 Needs Resource hold by P_3

04 P_3 Needs Resource hold by P_1

Prevention Conditions

Deadlock occurs when all four necessary conditions (Mutual Exclusion, Hold and wait, No Pre-emption and circular wait) happen together.

To prevent deadlock, we can break any one of these conditions.

1. Eliminate Mutual Exclusion,

1) Mutual Exclusion means only one process can use resource at a time.

2) Make resources sharable where possible, so multiple processes can use them simultaneously.

3) Example, Read-only files can be accessed by many processes at same time.

2. Eliminate Hold and wait.

1) This occurs when a process holds some resources and waits for others.

2) Ensure processes request all resources at the start. Alternatively, release held resources before requesting new ones.

3) Need scanner & printer both.

3. Eliminate No-Preemption

1) A process can't be forced to release resource using it.

2) Allow system to preempt resources from processes if necessary.

3) Priority-wise (Example).

4. Eliminate circular wait.

Assign priority numbers to resources.

Processes can only request resources in increasing order of priority.

Example P1 has priority 1, & P2 has priority 2. Process holding P2 can't request P1.

Bankers Algorithm

The Bankers Algorithm is a resource allocation and deadlock avoidance algorithm.

It is designed to simulate allocation of resources to multiple processes in way that ensures the system can avoid deadlock.

The algorithm is named so because it is analogous to banking system where a bank never allocates its available cash in such a way that it cannot satisfy needs of all its clients.

Steps

1) Calculate Need matrix.

Sub - Allocated resources from maximum requirement for process.

2) check Safe Sequence

o Start with available Resources.

o Look for process whose Need is less than equal to Available resources.

o If such a process is found,

↳ Assume it runs and releases its allocated resources

↳ Add these resources to Available resources

↳ mark process as finished.

o Repeat this process for all process

②) Safe State: If all process can finish in some order, the system is in safe state. If not, unsafe state, deadlock can occur.

Given

Available resources (3, 3, 2)

Process	Maximum	Allocation	(max) Need
P ₀	(7, 5, 3)	(0, 1, 0)	(7, 4, 3)
P ₁	(3, 2, 2)	(2, 0, 0)	(1, 2, 2)
P ₂	(9, 0, 2)	(3, 0, 2)	(6, 0, 0)

Calculate Need matrix

Safe Sequence

P₁'s Need: (1, 2, 2) is $\leq$ Available (3, 3, 2)

P₁ can finish and release resources.

New Available = (3, 3, 2) + (2, 0, 0) = (5, 3, 2)

P₀'s Need: (7, 4, 3) is $\leq$ (3, 3, 2)

It can't be executed.

P₂'s Need = (6, 0, 0) is $\leq$ (5, 3, 2)

P₂ can finish the resources

(5, 3, 2) + (3, 0, 2) = (8, 3, 4)

P₀'s Need = (7, 4, 3) $\leq$ (8, 3, 4)

can finish

(8, 3, 4) + (0, 1, 0) = (8, 4, 4)

P₁'s Need: (1, 2, 2) is $\leq$ Available (8, 4, 4)

P₁ can finish

Safe Sequence

P₁ → P₂ → P₀

3.3 Types of Scheduling algorithms: First Come First Serve (FCFS), Shortest Job First (SJF), Shortest Remaining Time Next (SRTN), Round Robin (RR), Priority Scheduling, Multilevel Queue Scheduling

b) First Come First Serve (FCFS)

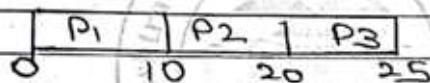
- 1) It is a non-preemptive scheduling algorithm which serve processes on queue and execute them.
- 2) Process which come first is allocate CPU first.
- 3) Real-life example, for ticket booking there is queue we follow that queue who comes first that gets ticket first.
- 4) Same way, in scheduling which process comes first get execute and exited first.
- 5) In multilevel Batch Queue, this scheduling algorithm is used for Background Queue.
- 6) Based on Arrival time means when the process has come based on that it execute.

P.T.O

Example: There are processes given with arrival and Burst time, calculate Average Turnaround and waiting time, with FCFS.

Process	Arrival time	Burst time
P ₁	0	10
P ₂	1	10
P ₃	2	5

Gantt chart



Completion Time

$$P_1 = 10 \quad P_2 = 20 \quad P_3 = 25$$

Turnaround Time (C.T - A.T)

04 (Completion Time - Arrival Time)

$$P_1 = 10 - 0 = 10$$

$$P_2 = 20 - 1 = 19$$

$$P_3 = 25 - 2 = 23$$

Average turnaround time

$$\frac{P_1 + P_2 + P_3}{3} = \frac{10 + 19 + 23}{3}$$

$$= 17.33 \text{ ms/unit}$$

Waiting Time (TAT - B.T)

(Turnaround time - Burst time)

$$P_1 = 10 - 10 = 0$$

$$P_2 = 19 - 10 = 9$$

$$P_3 = 23 - 5 = 18$$

Average waiting time

$$\frac{P_1 + P_2 + P_3}{3} = \frac{0 + 9 + 18}{3}$$

Question: Explain Round Robin algorithm with Suitable example.

⇒ Each process is assigned a small, fixed amount of CPU time called time Quantum or time slice.

The ready processes are stored in circular Queue, and CPU goes through one by one.

If a process doesn't finish in its time Quantum, it gets preempted (stopped) and placed at the back of Queue to wait for another turn.

Every process gets an equal chance to use CPU, making it fair and responsive.

It follows concept of context switching.

		Process		A-T	B-T	(Time Quantum = 3ms)
Ex		P11		0	8	P11 P12 P13 P14
	7	P12		1	4	0 3 6 9 12
	63	P13		2	9	P11 P12 P13 P14 P11
	2	P14		3	5	10 15 16 19 21 23
				P11 = 23 P13 = 26		P13 196
				P2 = 16 P14 = 21		